FlashSAC (arXiv:2604.04539): 분석 리포트

arXiv:2604.04539v2 (2026-05-15 제출) · RSS 2026 (arXiv 댓글 기준) · 공식 코드 MIT, 커밋 87edc90 (2026-04-09) · 분석일 2026-11-12
· 논문 그림은 © 저자, CC BY 4.0

핵심 요약

1. **무엇을 주장하나:** SAC에 큰 모델(액터·비판자 약 2.5M, 액터+비판자 앙상블 기준)과 큰 배치(2048), 병렬 환경 1024개를 쓰고 업데이트는 새 전이 1024개당 2회로 줄입니다. 비판자는 가중치 단위 노름 투영, 교차 배치 BN, RMSNorm, 분포형 비판자로 안정시킵니다. 본문은 GPU 25과제에서 PPO(200M step)와 FastTD3(50M step)를, CPU 40과제와 시각 8과제에서 기준선을 이긴다고 말합니다.
2. **공개 CSV로 확인한 것:** 고차원 10과제 전부에서 FlashSAC(50M)가 PPO(200M) 평균보다 높고, Holm 보정 후 유의합니다(10/10). 전체 25과제에서는 22과제에서 높습니다. 같은 50M에서 PPO와 비교해도 22과제에서 앞섭니다(`03` §2).
3. **확인하지 못한 것:** 벽시계 주장(시간 기준 "수 시간에서 수십 분", "3배 계산량")은 CSV에 시간 열이 없고 코드에 측정 코드도 없어 검증할 수 없습니다. 실물 G1 학습은 시드 1개이고 배포 코드가 없습니다.
4. **논문과 자료가 어긋나는 곳:** 조건수는 구조를 더할 때마다 줄지 않습니다(중앙값 기준 잔차 블록, RMSNorm 추가 시 증가). 기울기 노름은 최종 두 구조에서 0으로 기록되어 판단할 수 없습니다. 시각 1M 비교에서 MR.Q는 10과제 중 7과제에서 FlashSAC보다 높습니다(`03` §5, §7).
5. **코드와 표가 다른 곳:** UTD를 표 9는 2/2048, 본문과 코드는 2/1024로 적습니다. IsaacLab 스크립트만 n-step 3을 씁니다(표 9는 1). CPU 과제 수는 본문 40, 표 39. 실물 학습 하드웨어는 본문 RTX 5090, 실물은 A100으로 적힙니다(`04` D4, D23, `01` §6).
6. **액터 갱신의 BN 처리:** 공식 코드는 액터 갱신 때 다음 상태를 배치에 넣고, 비판자는 이동 평균 모드로 평가합니다. 논문에는 없는 선택입니다(`04` D10, D11).
7. **구현 판정:** 공식 GPU 재현은 이 맥(CUDA 없음)에서 불가능합니다. 공식 CPU 재현은 패키지 설치 승인 후 과제 일부만 조건부로 가능합니다. 알고리즘 재구현과 토이는 CPU에서 가능합니다. 실물과 시각 설정은 코드가 없어 재현 불가입니다(`04` §4).

이 리포트의 그래프 가운데 fig_로 시작하는 그림은 공개 CSV와 토이로 직접 그렸습니다. 논문 그림(paper_로 시작)은 저자 CC BY 4.0 표기를 붙였습니다.

FlashSAC 논문 분석 (arXiv:2604.04539)

0. 메타 정보

항목	내용
제목	FlashSAC: Fast and Stable Off-Policy Reinforcement Learning for High-Dimensional Robot Control
저자	Donghu Kim, Youngdo Lee, Minho Park, Kinam Kim, I Made Aswin Nahendra, Takuma Seno, Sehee Min, Daniel Palenicek, Florian Vogt, Danica Kragic, Jan Peters, Jaegul Choo, Hojoon Lee (공동 1저자 Kim·Lee, 교신 Choo·Lee)
학회	RSS 2026. arXiv 초록 페이지의 Comments란에 "RSS'26"이라고 적혀 있다. 학회 공식 목록은 확인하지 않았다
버전	v1 (2026-04-06, PDF 날짜 2026-04-07, 40쪽) → v2 (2026-05-15 제출, PDF 날짜 2026-05-18, 42쪽). 최신은 v2
라이선스	arXiv abs 페이지 `licenses/by/4.0/` 링크: **CC BY 4.0**. PDF 첫 쪽과 쪽 머리말에 다른 권리 표기가 없다(§9 참조)
공식 코드	https://github.com/Holiday-Robot/FlashSAC . 이 폴더의 `repo/` 에 얕게 복제(`--depth 1`), 커밋 `87edc90` (2026-04-09, "Create LICENSE"). 라이선스 MIT. 저장소에 `results/` 학습 곡선 CSV 47개가 있다
프로젝트 페이지	https://holiday-robot.github.io/FlashSAC/
분량	본문 1-12쪽(참고문헌 13-19쪽), 부록 A-E 20-42쪽

1. 한 문단 요약

FlashSAC는 SAC를 기반으로 한 off-policy RL로, 고차원 로봇 제어(손 조작, 휴머노이드)에서 PPO보다 빠르고 안정적으로 학습한다고 주장한다. 방법은 두 축이다. 첫째, 병렬 환경 1024개로 데이터를 많이 모으고, 큰 모델(2.5M 파라미터 급)과 큰 배치(2048)를 쓰되 업데이트 수를 크게 줄인다(UTD 2/1024, §4.1). 둘째, 비판자(critic)의 업데이트를 안정시키려고 가중치 노름을 단위 구면에 투영하고, 역잔차 블록과 배치 정규화(교차 배치), RMSNorm, 분포형 비판자와 보상 정규화를 쓴다(§4.2). 탐색은 엔트로피 목표를 행동 표준편차 σ=0.15로 정하고, 잡음을 Zeta 분포 길이만큼 반복한다(§4.3). 주장된 결과는 GPU 시뮬레이터 25과제에서 PPO(200M step)와 FastTD3(50M step)를 앞서고, CPU 시뮬레이터 40과제와 시각 8과제에서 기준선보다 낫고, 실물 G1 휴머노이드 학습 시간을 수 시간에서 수십 분으로 줄인다는 것이다.

2. 문제 설정과 동기 (Sec. 1-2)

- 시뮬레이션 기반 RL은 손 조작(고차원 상태-행동)에서 on-policy PPO의 데이터 효율이 떨어진다. PPO는 매 반복마다 데이터를 버린다 (p.1-2).
- off-policy는 재생 버퍼로 데이터를 재사용하지만, 고차원에서는 비판자를 맞추는 데 업데이트가 많이 필요하고, 부트스트랩 오차가 쌓인다 (식 1, p.2).
- 기존 연구를 세 갈래로 나눈다 (Sec. 2.2, p.3-4): (a) 속도: 병렬 시뮬레이션과 큰 버퍼 (FastTD3, FastSAC; 작은 네트워크 약 0.2M 파라미터), (b) 안정성: 가중치·특징·기울기 노름 제한, 재초기화, 증류, 앙상블, 대체 표적망, (c) 탐색: OU 잡음, 핑크 잡음, 파라미터 잡음, 상태 의존 탐색, 행동 반복.
- 논문의 차별점은 세 갈래를 한 알고리즘에서 함께 다룬다는 것이다 (p.3 마지막 문단).

3. 방법 (Sec. 3-4)

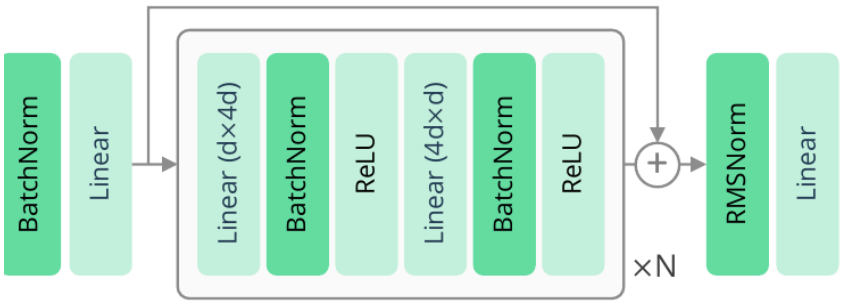
3.1 기준선 SAC (Sec. 3.2, 식 2-5)

이중 비판자와 최소 목표, 엔트로피 항, 지수이동평균 표적망 τ를 그대로 쓴다. 식 (4)는 MSE 형태로 적혀 있지만, §4.2는 분포형 비판자로 교차 엔트로피를 쓴다고 명시한다. 이 차이는 식 표기 문제이고, 코드는 분포형 구현이다(04 D1).

3.2 빠른 학습 (Sec. 4.1)

요소	논문 값	출처
병렬 환경	1024	p.5
재생 버퍼	최대 10M 전이(표준 1M의 10배)	p.5
네트워크	액터·비판자 6층, 약 2.5M 파라미터, 배치 2048	p.5
UTD	"2/1024": 새 전이 1024개당 업데이트 2회 (본문). 표 9는 "2/2048"로 적힘	p.5, 부록 Table 9 (§6 6번)
정밀도	혼합 정밀도 (5-10% 가속이라고 주장)	p.5

논문은 "데이터·모델 확장과 업데이트 감소"가 supervised scaling law에서 왔다고 설명한다(p.5, [37]). 이 주장을 검증하는 것은 §6.2 소거 실험이다.



논문 Fig. 2: 역잔차 블록(BN → ReLU, 잔차) 뒤 RMSNorm과 선형 헤드. © 저자, CC BY 4.0. 원문 https://arxiv.org/abs/2604.04539

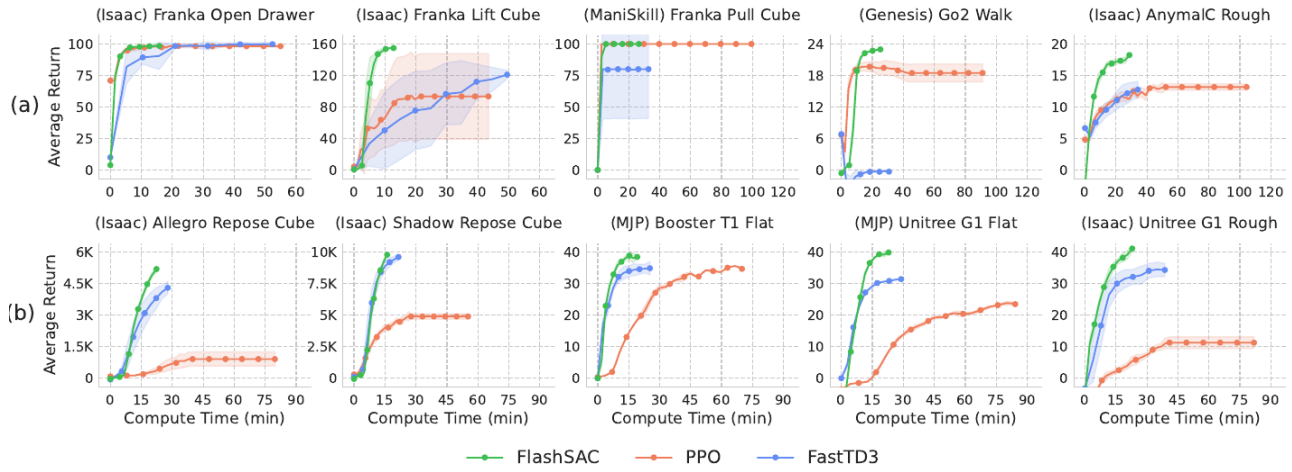
3.3 안정 학습 (Sec. 4.2)

1. **역잔차 블록**: 확장 4배 (d→4d→d), 블록 뒤 잔차 연결. 블록 수 2 (p.5-6).
2. **RMSNorm** 백본 뒤: 특징 노름을 제한해 비판자 입력이 발산하지 않게 함 (p.6).
3. **배치 정규화(BN), ReLU 앞**: 재생 데이터의 비정상 분포에서 활성이 포화되는 것을 막고, 큰 배치 통계로 손실 지형의 조건수를 낮춘다고 주장. LayerNorm 대신 BN을 고른 이유로 [83, 8, 72] 인용 (p.6).
4. **교차 배치 값 예측**: 현재 상태와 다음 상태를 한 배치로 묶어 BN 통계를 공유 (CrossQ의 방법을 따름, p.6).
5. **분포형 비판자와 보상 정규화**: 101개 원자, 지지 [-5, 5], 교차 엔트로피 손실. 보상은 식 (6)으로 $\bar{r} = r / \max(\sqrt{(\sigma^2_G + \varepsilon)}, G_max,t / G_max)$로 정규화 (p.6).
6. **가중치 정규화**: 매 기울기 갱신 뒤 각 가중치 벡터를 단위 노름으로 투영, (γ, β)는 노름 √d로 투영 (p.6). PDF 추출에서 √ 기호가 사라져 "norm d"로 보인다(04 D8).

3.4 탐색 (Sec. 4.3)

1. **통일된 엔트로피 목표**: $\bar{H} = \frac{1}{2}|A| \log(2\pi e\, \sigma^2_tgt)$, σ_tgt = 0.15 (식 7). 행동 차원에 따라 자동으로 스케일된다 (p.6).
2. **잡음 반복**: 정규분포 잡음 ε을 Zeta 분포 P(k) ∝ k^(-s) (s=2, 최댓값 16) 길이만큼 고정한다. 병렬 환경에서 환경마다 OU·핑크 잡음 상태를 두지 않는 대신, 짧은 로컬 상태만 쓴다 (p.6-7). 코드는 반복 길이를 모든 환경이 공유하는 스칼라 카운터로 세므로, 환경 간 잡음 갱신 시점이 같다 (04 D15).

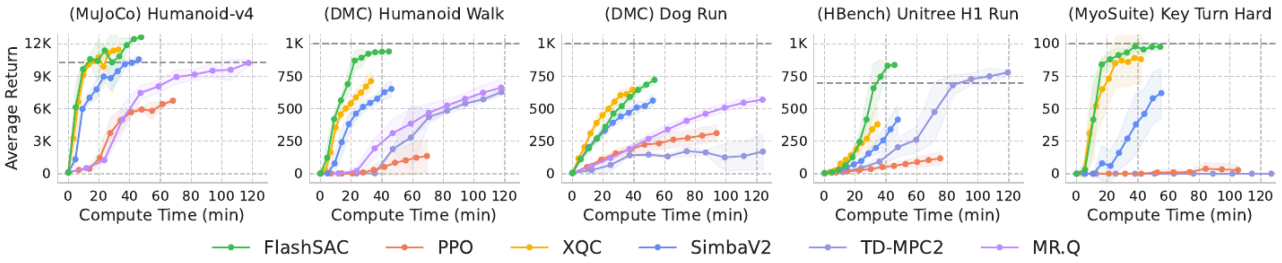
4. 실험 (Sec. 5, 부록 E)



논문 Fig. 3: GPU 시뮬레이터 대표 과제의 학습 곡선(벽시계 축). FlashSAC, PPO, FastTD3. © 저자, CC BY 4.0. 원문 https://arxiv.org/abs/2604.04539

4.1 GPU 시뮬레이터, 상태 기반 (Sec. 5.1, Fig. 3, 부록 E.1-E.4)

- 과제 25개: 저차원 15 (Franka 잡기·서랍, ManiSkill 파지, Anymal-C/D, Go2, Genesis 파지), 고차원 10 (Allegro·Shadow 손 재배치, G1·H1·T1 보행). 부록 Table 1-4가 정리하고, 공개 스크립트에 있는 UnitreeG1 두 과제는 표에 없다(§6 5번).
- 기준선: PPO(RSL-RL, 200M step), FastTD3(50M step). FlashSAC와 FastTD3는 50M step.
- 주장: 저차원에서는 PPO와 비슷하거나 약간 낮고, 고차원에서는 "수렴 점수가 높고 벽시계 시간이 훨씬 적다". FastTD3는 Go2 Walk, Franka Pull Cube에서 자주 실패한다.
- 설정: 단일 설정, 감마만 시뮬레이터 기본값(IsaacLab 0.99, Playground 0.97)으로 바꿈 (p.7). 공개 스크립트는 IsaacLab에서만 n-step을 3으로 둔다(§6 6번).
- 벽시계는 RTX 5090 1장으로 측정했다고 본문에 있다 (p.7). 부록 A는 AMD Ryzen 9 9950X3D + RTX 5090이라고 적는다.



논문 Fig. 4: CPU 시뮬레이터 대표 과제의 학습 곡선(벽시계 축). © 저자, CC BY 4.0. 원문 https://arxiv.org/abs/2604.04539

4.2 CPU 시뮬레이터, 상태 기반 (Sec. 5.2, Fig. 4, 부록 E.5-E.8)

- 과제 "40개"(본문): MuJoCo 5, DMC 10, MyoSuite 10, HumanoidBench 14 (부록 표 합계는 39, §6 5번).
- 기준선: PPO, XQC, SimbaV2, TD-MPC2, MR.Q.
- 설정 차이: 배치 512, UTD 1, 환경 1개 (p.8). 주장: "모든 기준선을 일관되게 앞선다".

4.3 시각 기반 (Sec. 5.3, Fig. 5, 부록 E.9)

- 본문은 DMC 8과제, 기준선 DrQ-v2와 MR.Q (p.9). 인코더 3층 합성곱, 프레임 스택 3, 3단계 반환, 액션 반복 2, 1M step.

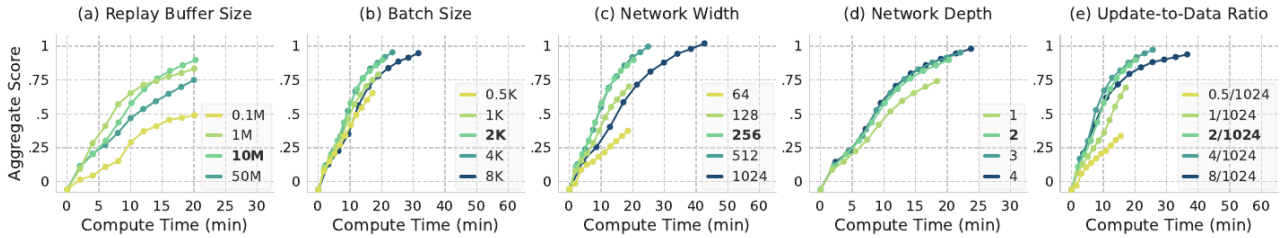
- 주장: "대부분 과제에서 점근 성능과 벽시계 효율이 기준선과 같거나 높다". 공개 저장소에는 픽셀 학습 코드가 없다(04 D26).

4.4 실물 G1 휴머노이드 (Sec. 5.4, Fig. 1c·Fig. 6)

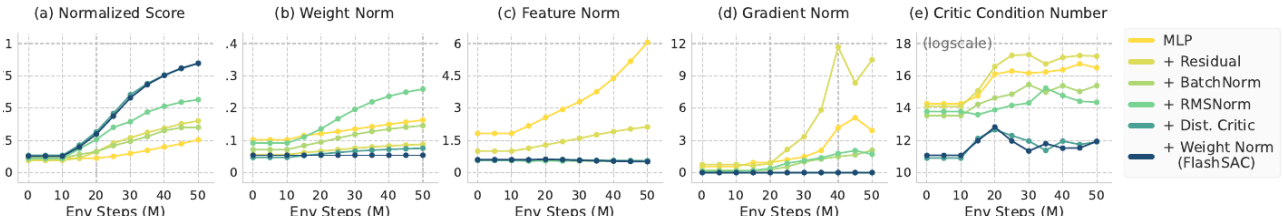
- 블라인드 보행, 지형 커리큘럼, 29-DoF G1. 도메인 무작위화. 평지: 약 20분(FlashSAC) 대 약 3시간(PPO). 계단(15 cm, 1.5 m 플랫폼): 약 4시간 대 약 20시간 (p.9-10).
- 같은 보상과 같은 위치 추정 기법(context estimator, asymmetric critic)을 두 방법에 쓴다고 한다 (p.9-10).
- 부록 D.0.1: 학습에 A100 1장, 4096 환경, 약 4시간. 본문과 하드웨어가 다르다(§6 10번).

5. 분석 (Sec. 6, Fig. 7-10)

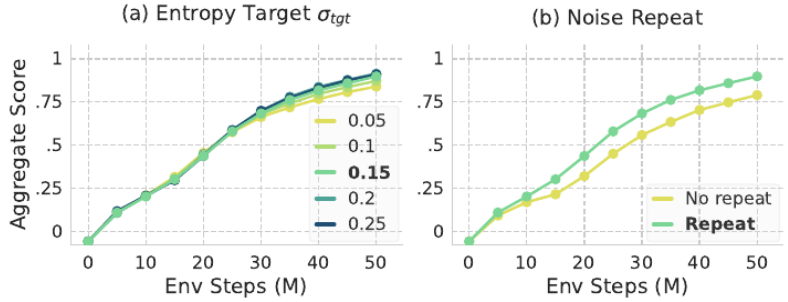
- **6.1 on/off-policy 분포**(Fig. 7): Shadow Hand 1M step 학습 후 재생 버퍼와 최종 정책 롤아웃 1M의 2D 밀도를 비교. 재생 버퍼가 훨씬 넓다. 결론: on-policy 데이터는 좁아 고차원에서 불리하다. 과제 1개, 시드 수 미기재.
- **6.2 확장 소거**(Fig. 8, 벽시계 x축): 재생 버퍼 크기(0.1M, 1M, 10M, 50M), 배치(0.5k-8k), 폭(critic 64-1024), 깊이(1-4블록), UTD(0.5-8/1024). 주장: 버퍼는 10M이 최적, 나머지는 "배치와 용량을 키우고 UTD를 줄이면 빨라진다".
- **6.3 구조 소거**(Fig. 9): MLP → +잔차 → +BN → +RMSNorm → +분포형 → +가중치 정규화. 주장: 매 추가가 가중치·특징·기울기 노름을 제한하고 조건수를 낮추며, 조건수는 단조 감소하고 최종 구조에서 가장 낮다.
- **6.4 탐색 소거**(Fig. 10): σ_tgt ∈ {0.05, 0.1, 0.15, 0.2, 0.25}, 잡음 반복 유무. 주장: σ_tgt는 0.15-0.2가 최적이고, 반복을 끄면 느리고 낮다.



논문 Fig. 8: 재생 버퍼, 배치, 폭, 깊이, UTD의 소거(벽시계 축, 집계 점수). © 저자, CC BY 4.0. 원문 https://arxiv.org/abs/2604.04539



논문 Fig. 9: 구조 소거와 노름·조건수 추적. 조건수 축은 로그(ln 추정). © 저자, CC BY 4.0. 원문 https://arxiv.org/abs/2604.04539



논문 Fig. 10: 엔트로피 목표 σ_tgt와 잡음 반복의 소거. © 저자, CC BY 4.0. 원문 https://arxiv.org/abs/2604.04539

6. 읽으면서 본 문제와 한계

이 절은 논문이 말하지 않거나 근거가 약한 것이다. 공개 CSV와 코드로 확인한 결과는 03과 04에 있다.

1. **벽시계 주장의 측정 방식이 불투명하다.** 부록 A는 환경 시간과 업데이트 시간을 합친다. 업데이트 시간은 MuJoCo Humanoid-v4(상태)와 DMC Walker-run(시각) 한 과제에서 한 번 재고 전 과제에 재사용한다. 그래서 같은 알고리즘의 과제별 차이와 기준선(PPO 등)의 업데이트 비용은 반영되지 않는다. 공개 CSV에는 시간 열이 없어서 벽시계 곡선은 재계산할 수 없다(03 §1).
2. **비교 예산이 다르다.** PPO는 200M step이고 FlashSAC는 50M이다. 논문은 PPO 계산량이 약 3배라고 쓴다(p.7). 같은 env step 50M에서 비교하면 격차는 줄지만 대부분 과제에서 남는다(03 §3).
3. **시드 수가 본문에 없다.** 그림 설명은 "random seeds"라고만 쓴다. 공개 스크립트는 시드 5개(0, 1000, 2000, 3000, 4000)를 쓰지만, PPO GPU는 0-5, TD-MPC2는 4개, 실물 실험은 1개다(03 §1).
4. **정규화 점수의 정의가 없다.** MuJoCo는 TD7 기준치로 정규화한다고 하지만 random 점수 보정이 있는지, 식이 무엇인지 적혀 있지 않다. 이 분석은 반환 / Normalize Score로 가정했다.
5. **과제 개수와 목록이 맞지 않는다.** GPU 본문 25과제(표와 일치). 공개 CSV에는 27과제가 있고, 추가된 UnitreeG1PlaceAppleInBowl·TransportBox는 ManiSkill 스크립트에 있지만 표에는 없다. CPU 본문 40과제, 표 합계 39과제, CSV 39과제. 시각 본문 8과제, CSV는 DMC 10과제와 Adroit형 3과제.
6. **UTD와 n-step 표기가 코드와 다르다.** 표 9는 UTD 2/2048과 n=1이다. 코드의 IsaacLab 스크립트는 UTD 2/1024(본문과 같음)와 n=3을 쓴다(04 D4, D23).
7. **"기울기 노름을 명시적으로 제한한다"는 말이 과하다.** 저장소에는 기울기 클리핑이 없다(검색 0건). 기울기 노름은 가중치 투영을 통해 간접적으로만 제한된다(04 D19).
8. **조건수 단조 감소 주장이 공개 데이터에서 성립하지 않는다.** 잔차 블록을 넣으면 조건수 중앙값이 1.4e7에서 3.3e7로 커진다. 마지막 구조가 중앙값 기준으로 가장 낮긴 하지만(8.7e4), 단조는 아니다(03 §7). 기울기 노름은 분포형 비판자와 최종 구조에서 0으로 기록되어 이 지표로 판단할 수 없다.
9. **실물 실험은 시드 1개다.** 평지 실험에서 두 방법의 시작 지형 단계값이 다르다(FlashSAC 3.47, PPO 3.41). 이 지표로 "시간 비교"를 하기 어렵다. 거친 지형에서는 같은 env step에서 FlashSAC가 PPO보다 빠르게 3단계에 도달했다(03 §6).
10. **하드웨어가 여러 곳에서 다르게 적혀 있다.** 본문 RTX 5090, 부록 A 9950X3D + RTX 5090, 실물 학습 A100. 실물 학습의 환경 수 4096은 GPU 설정 1024와 다르고, 부록 D.0.1은 "5.2의 CPU 설정과 같은 구조와 하이퍼파라미터"라고 한다. 어느 설정인지 불분명하다.
11. **가치 오차 진단이 없다.** 비판자의 과대추정을 직접 재지 않는다. 토이에서 이 지표를 넣었다(03 §9).
12. **소거 실험은 env step 축으로만 공개 자료가 있다.** 소거 실험은 IsaacLab 4과제, 시드 5개다. 배치 4k·8k, 폭 1024, 블록 4, UTD 4·8이 기본값과 같거나 더 높다. 기본값은 계산 비용을 고려한 선택으로 보인다. 벽시계 축이 없어서 이것을 확인할 수 없다(03 §7).

7. 주요 주장 대조표

주장	위치	공개 자료로 확인	판정
고차원 과제에서 PPO보다 최종 성능이 높다	§5.1, Fig. 3b	10/10 과제에서 FlashSAC(50M) 평균이 PPO(200M)보다 높고, 10/10이 Holm 보정 유의	확인 (03 §2)
저차원 과제에서 PPO와 비슷하거나 약간 낫다	§5.1, Fig. 3a	15과제 중 12과제에서 평균이 높고, 6과제가 유의. PokeCube는 크게 뒤짐	대체로 확인 (03 §2)
FastTD3는 몇 과제에서 자주 실패한다	§5.1	7과제에서 시드 하나 이상이 FlashSAC 평균의 절반 미만. Go2 Walk·PegInsertion·RollBall은 5시드 모두	확인 (03 §2)

주장	위치	공개 자료로 확인	판정
CPU 40과제에서 모든 기준선을 앞선다	§5.2	1M step 비교에서 39과제 중 SimbaV2 30, XQC 30, TD-MPC2 31, MR.Q 13(15 중) 에서 평균 우위	대체로 확인, 모든 과제는 아님 (03 §4)
시각 8과제에서 기준선과 같거나 높다	§5.3	1M step 비교에서 DrQ-v2 9/10 우위, MR.Q는 3/10 우위(7/10에서 MR.Q가 높음)	부분 반박 (03 §5)
실물 평지 약 20분 대 3시간, 계단 4시간 대 20시간	§5.4	벽시계 없음. 계단 지형 단계 3.0 도달 env step 비율 약 7.8배(시드 1)	검증 불가(시간), 경향만 확인 (03 §6)
배치·UTD를 키우면 빨라진다	§6.2	env step 기준으로 배치 4k·UTD 4 이상은 기본값과 같거나 높다	벽시계 없이는 판단 불가 (03 §7)
조건수는 구조 추가마다 감소하고 최종 구조가 최저	§6.3	중앙값 기준 최저는 최종 구조. 단조는 아님	부분 반박 (03 §7)
탐색 σ_tgt 0.15-0.2가 최적, 잡음 반복이 중요	§6.4	0.2 +0.016, 0.25 +0.012, 0.05 −0.062; 반복 끄면 −0.115	확인 (03 §7)

8. v1 → v2 차이

텍스트를 문장 단위로 대조했다(인용 번호를 지운 뒤 비교). 실험 수치, 과제 목록, 부록 표의 값, 하이퍼파라미터 문장은 같다. 바뀐 부분은 다음과 같다.

위치	v1	v2
날짜·쪽 수	2026-04-07, 40쪽	2026-05-18, 42쪽 (arXiv v2 2026-05-15)
관련 연구 2.2 첫 문단 (도전 과제 수)	느림과 불안정 두 가지	고차원 행동 공간에서의 탐색을 세 번째 도전으로 추가
관련 연구 2.1	PPO 한 문단	신뢰 영역 개선(PPO 계열), 행동 공간 확장 탐색, 경로별 추정 분산 감소 등 두 문장 추가
관련 연구 2.2	FastTD3 (고차원에서 빠름)	FastTD3와 FastSAC 함께 언급
관련 연구 2.2 안정화	가중치, 특징, 기울기 노름	재초기화, 증류, 앙상블, 대체 표적망 목록 추가
관련 연구 2.2 탐색	없음	탐색 셋째 갈래(OU, 핑크, 파라미터 잡음, 상태 의존 탐색, 행동 반복) 추가
관련 연구 2.2 마지막 문단	속도와 안정 두 방향을 묶는다고 씀	탐색을 더해 세 방향(속도, 안정, 탐색)을 묶는다고 씀
참고문헌·번호	인용 번호 다름	재번호

v2에서 추가된 문장은 주로 관련 연구와 탐색 설명이다. 실험 결과나 하이퍼파라미터를 바꾸는 내용은 찾지 못했다.

9. 그림 저작권과 공개 여부

- arXiv abs 페이지: CC BY 4.0. PDF 첫 쪽·쪽 머리말·마지막 쪽에 다른 권리 표기가 없다(`pdftotext`로 확인, "©", "copyright", "license" 검색 결과 없음).
- 그림 출처: Fig. 3, 4, 5, 8, 9, 10 등 그래프는 저자가 그렸다. Fig. 2는 구조도다. Fig. 1은 여러 시뮬레이터 장면과 실로봇 사진을 합쳐 만든 그림이고, Fig. 6은 실로봇 사진이다. 두 그림에는 제3자 장면 또는 제품 사진이 들어 있어, 출처가 없는 상태에서 공개하지 않는다(보수적 판정).
- 판정: 그래프와 구조도(Fig. 2, 3, 4, 8, 9, 10)는 "© 저자, CC BY 4.0. 원문 https://arxiv.org/abs/2604.04539"를 붙여 공개 리포트와 게시본에 넣는다. Fig. 1과 Fig. 6은 이 분석에서 싣지 않았다(공개 불가 판정이므로 개인용으로도 자르지 않음). Fig. 5(시각 학습 곡선)와 Fig. 7(밀도 그림)은 본문 설명으로 충분해서 싣지 않았다.

10. 논문의 핵심 기여를 한 줄로

고차원 로봇 제어에서 SAC의 느린 수렴과 불안정을, "데이터와 모델을 키우고 업데이트를 줄이되, 비판자의 노름과 정규화를 구조적으로 제한한다"는 설계로 함께 다룬 실험적 연구다. 방법의 개별 요소는 기존 연구(CrossQ, SimbaV2/HyperSpherical, FastTD3, Zeta 반복 등)에서 왔고, 기여는 조합과 대규모 평가에 있다(02 §1).

FlashSAC 선행연구와 계보

1. 계보 한눈에

```
DPG (결정적 정책 경사)
 └ DDPG (재생 버퍼, 표적망, OU 잡음)              [1509.02971]
    └ TD3 (이중 비판자, 지연 갱신, 목표 잡음)    [1802.09477 계열]
       └ SAC (최대 엔트로피, 자동 온도)           [1801.01290]
          ├ REDQ·DroQ 계열 (앙상블, 드롭아웃)
          ├ CrossQ (BN + 표적망 없이 교차 배치)  [ICLR 2024, 논문 [8]]
          ├ SimBa, SimbaV2 (단순성 편향, 초구면 정규화) [논문 [44], [45]]
          ├ XQC (BN 기반 비판자, 높은 표본 효율)   [논문 [72]]
          ├ 병렬 시뮬레이션 계열: FastTD3 [88], FastSAC [87] (약 0.2M 파라미터)
          └ FlashSAC (이 논문): 위 요소를 한 알고리즘에 합친다
on-policy 기준선: PPO [85, 86] (RSL-RL 구현으로 비교)
모델 기반·표현 계열: TD-MPC2 [24], MR.Q [19], DreamerV3 [23]
시각 기준선: DrQ-v2 [111] (DDPG 계열 + 이미지 증강), DrM
```

논문 저자 가운데 Hojoon Lee, Jaegul Choo, Takuma Seno, Donghu Kim는 SimBa(논문 [44])와 초구면 정규화(논문 [45])의 저자이기도 하다. 그래서 가중치 투영과 BN 선택은 그 계열의 연장선으로 읽을 수 있다. 이 분석은 저자 소속을 다루지 않는다.

2. 가까운 대안과 차이

방법	무엇을 바꾸나	FlashSAC와 다른 점	논문에서의 위치
SAC (기준)	최대 엔트로피, 이중 비판자	얕은 MLP, 배치 256, UTD 1, 1M 버퍼	§3.2
CrossQ [8]	BN과 교차 배치(논문 기술 기준)	FlashSAC는 표적망(τ 0.01)을 유지하고, 가중치 투영과 RMSNorm, 분포형 비판자를 추가	§4.2 "교차 배치"는 CrossQ를 따른다고 명시
SimbaV2 [45]	초구면 정규화(가중치와 특징을 단위 노름으로 제한)	같은 계열. 세부 구조(BN 사용 여부, 액터 적용 범위)는 이 분석에서 확인하지 않았다	§2.2
XQC [72]	BN 기반 비판자, 높은 표본 효율	같은 계열. 비교는 CPU 설정(배치 512, UTD 1)에서만 한다	§5.2
FastTD3 [88]	병렬 환경, 작은 네트워크, 빠른 벽시계	같은 병렬 환경 계열이지만 큰 모델·큰 배치·적은 업데이트를 쓴다	§5.1 (GPU 기준선)
FastSAC [87]	병렬 환경에서 SAC 고속화	같은 계열. 공개 CSV는 없다	§2.2
PPO [86] (RSL-RL)	on-policy, 큰 병렬 샘플	200M step, 공개 CSV에 비교 곡선 있음	§5.1, §5.4
TD-MPC2 [24]	잠재 세계 모델과 MPC	세계 모델 비용 때문에 벽시계 효율이 낮다고 주장	§5.2
MR.Q [19]	모델 기반 보조 목표로 표현 학습	FlashSAC는 보조 목표를 쓰지 않는다. 논문은 둘이 결합 가능하다고 말함	§5.3
DrQ-v2 [111]	DDPG 계열 + 이미지 증강	시각 과제에서 FlashSAC가 더 안정하다고 주장	§5.3
DDPG [51]	결정적 정책, 표적망	기준선 아님. 이 저장소의 DDPG 분석에서 은닉층 전체 BN이 LQR 토이에서 성능을 떨어뜨렸다(§3 참조)	§2.2

3. 이 저장소의 다른 논문 분석과의 관계

아래 연결은 해당 논문 분석 노트의 결론을 인용한 것이다. 각 노트는 독립적으로 읽을 수 있다.

- **DDPG 분석 (arXiv:1509.02971).** 같은 off-policy actor-critic 계열이다. 그 분석의 LQR 토이에서 은닉층 전체 BN은 표적 망의 유무와 관계없이 성능을 떨어뜨렸다. FlashSAC는 은닉층마다 BN을 넣고 교차 배치 통계를 쓴다. 두 결과가 맞지 않아 보인다. 차이는 설정(DDPG 토이는 배치 64, 64-64 망, 결정적 정책)에서 올 수 있다. 이 분석은 SAC 토이에서 BN 설정을 따로 시험했다(03 §9).
- **DexPolicy 분석 (arXiv:2610.00360).** PPO에서 행동 잡음 σ를 학습 중에 0.20에서 0.04로 줄이는 스케줄을 다룬다. FlashSAC는 잡음 크기를 엔트로피 목표(σ_tgt 0.15)로 고정하고, 대신 잡음의 시간 상관을 반복으로 만든다. 탐색 설계를 비교할 때 한 축은 "크기 스케줄"이고 다른 축은 "시간 상관"이다.
- **Foresight Residual RL 분석 (arXiv:2607.16506).** 보상에 확률 예측기를 곱하는 설계가 가치 오차에 민감하다는 결론이다. FlashSAC의 분포형 비판자는 예측 값을 지원 [-5, 5] 안에 묶는다. 값이 범위를 벗어나지 못하게 하는 방식이라 같은 문제를 다른 방식으로 다룬다.
- **ThinkWorld 분석 (arXiv:2610.37666).** 보조 예측 손실과 강화학습을 함께 쓰는 VLA 연구다. 그 분석은 보고된 이득의 대부분이 보조 예측 손실에서 나온다고 해석했다. FlashSAC도 보조 목표 없이 순수 RL로 비교하므로, MR.Q·TD-MPC2와의 비교는 "보조 목표 대 구조 정규화"의 대조로 읽을 수 있다.
- **MPM 기울기 신뢰성 분석 (arXiv:2609.34666).** 원자 덧셈 순서와 긴 롤아웃에서 기울기가 흔들린다는 결론이다. FlashSAC의 가중치 투영과 RMSNorm은 기울기와 특징 노름을 제한하는 수단이라, 수치 안정성 문제에 대한 학습 측의 대응으로 볼 수 있다. 같은 현상을 직접 측정한 것은 아니다.

4. 공개 구현 (GitHub)

2026-11-12, HTTP HEAD 요청으로 저장소 주소가 살아 있는지만 확인했다. 별 개수, 라이선스, 최근 push 날짜는 API를 쓰지 않아 확인하지 않았다. 라이선스는 공식 저장소의 LICENSE 파일만 읽었다.

저장소	성격	라이선스	주소 확인	대조
Holiday-Robot/FlashSAC	공식 (커밋 `87edc90`, 2026-04-09)	MIT (LICENSE 파일 확인)	200	04에서 정적 대조
younggyoseo/FastTD3	FastTD3 (논문 [88]으로 보이나 저자 확인 안 함)	미확인	200	기준선 설명만
FastSAC (논문 [87])	저장소 주소를 찾지 못함 (`younggyoseo/FastSAC` 404)	미확인	미확인	—
dojeon-ai/SimbaV2	SimbaV2 (논문 [45]와 이름 일치)	미확인	200	이름으로 연결
facebookresearch/mrq	MR.Q (논문 [19]와 이름 일치)	미확인	200	이름으로 연결
XQC (논문 [72])	저장소 주소 미확인	미확인	미확인	—
nicklashansen/tdmpc2	TD-MPC2 (논문 [24])	미확인	200	이름으로 연결
leggedrobotics/rsl_rl	PPO 구현 RSL-RL (논문 [86])	미확인	200	기준선 구현
facebookresearch/drqv2	DrQ-v2 (논문 [111])	미확인	200	시각 기준선
adityab/CrossQ	CrossQ (논문 [8])	미확인	200	교차 배치 원형
isaac-sim/IsaacLab	시뮬레이터 (논문 [61])	미확인	200	환경
google-deepmind/mujoco_playground	시뮬레이터 (논문 [112])	미확인	200	환경
haosulab/ManiSkill	시뮬레이터 (논문 [95])	미확인	200	환경
Genesis-Embodied-AI/Genesis	시뮬레이터 (논문 [4])	미확인	200	환경
MyoHub/myosuite	시뮬레이터 (논문 [9])	미확인	200	환경

저장소	성격	라이선스	주소 확인	대조
HumanoidBench (논문 [89])	저장소 주소 미확인	미확인	미확인	—
google-deepmind/dm_control	DMC (논문 [96])	미확인	200	환경
Farama-Foundation/Gymnasium	Gym 계열 (논문 [99])	미확인	200	환경

고르는 기준: 이 논문을 실제로 돌리려면 공식 저장소 하나면 된다. 기준선 비교는 PPO는 RSL-RL, 시각은 DrQ-v2 공식 저장소로 맞출 수 있다. FastSAC와 XQC, HumanoidBench의 저장소는 이 분석에서 찾지 못했으므로, 그 기준선은 논문 수치나 공식 CSV에 의존한다.

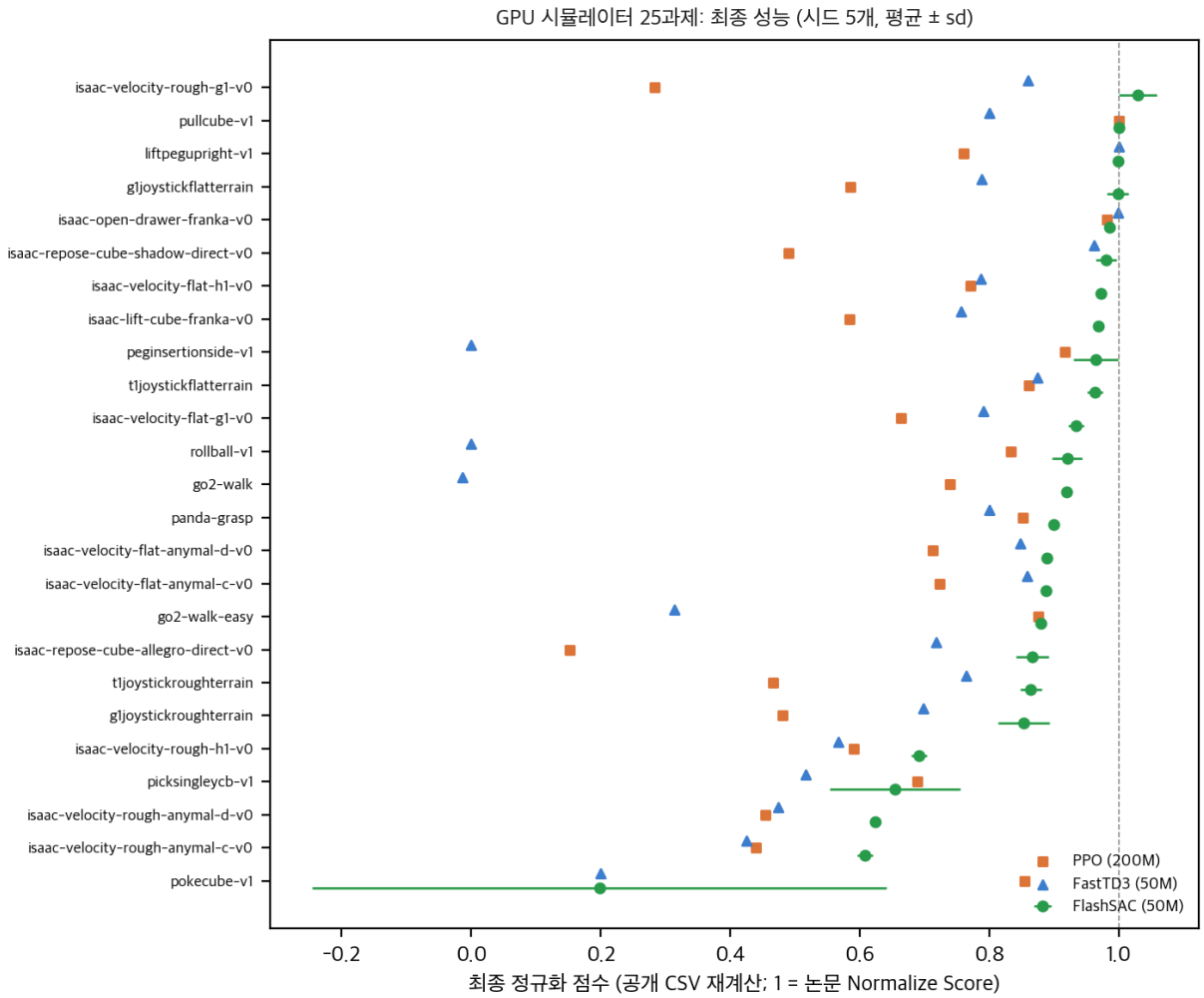
5. 이 분석에서 중요한 선행연구 질문

1. "작은 배치와 많은 업데이트"는 off-policy의 기본값이다. FlashSAC는 반대 방향(큰 배치, 적은 업데이트)을 택했다. 이 방향이 언제 이득인지는 벽시계 기준으로 따져야 한다. 샘플 기준으로는 큰 배치가 대체로 낫다(03 §7).
2. 가중치 노름 제약은 CrossQ, SimbaV2, FlashSAC가 공통으로 쓴다. 차이는 적용 위치(비판자만, 액터까지)와 BN과의 조합이다.
3. 분포형 비판자(C51 계열)는 값 범위를 제한하는 장점이 있지만, 지원 범위를 잘못 잡으면 값이 끝에 몰린다. 이 논문은 [-5, 5]를 정규화된 보상에 맞춰 골랐다(04 D17).

FlashSAC 수치 재계산과 검산

1. 재계산 설정

- **입력:** 공식 저장소의 `results/` CSV 47개(공개 학습 곡선). 열은 `env_step`, `value`, 지표 이름, 과제, 시드뿐이다. **벽시계 열이 없다.** 그래서 벽시계 주장은 아래에서 검증하지 않았다.
- **스크립트:** `analysis/recompute_results.py` → `analysis/recompute_results.json`. 부트스트랩 1만 번, 난수 시드 0.
- **정규화:** 반환 / Normalize Score (논문 부록 Table 1-8). ManiSkill과 MyoSuite는 성공률(0-1). 논문은 정규화 식을 쓰지 않았으므로 이것은 가정이다. MuJoCo에 random 점수 보정이 없다는 점도 같다.
- **최종 성능:** 시드별 마지막 기록 값의 평균. **같은 환경 step 비교:** 시드별 곡선을 선형 보간한 값. 기준 step은 GPU 50M, CPU·시각 1M.
- **유의성:** 과제마다 시드 값 두 집합에 Welch t 검정과 Mann-Whitney 검정을 하고, 평균 차의 95% 부트스트랩 구간을 냈다. 과제 전체에서 Holm 보정을 했다. 시드가 5개라 Mann-Whitney의 가능한 최소 p는 약 0.008이다. 해석은 Welch 기준으로 한다.
- **제한:** 시드 5개와 곡선 11점은 p값을 크게 흔든다. 아래 "유의"는 공개 CSV의 시드 안에서만 나온 값이다.



직접 계산: 과제별 최종 정규화 점수(시드 평균 ± sd). 기준선 PPO 200M·50M, FastTD3 50M.

2. GPU 시뮬레이터 (상태 기반), 25과제

비교는 FlashSAC 50M 대 PPO 200M(논문 설정 그대로), 그리고 같은 50M에서 본 PPO, FastTD3 50M이다.

그룹	과제 수	FlashSAC > PPO(200M)	Holm 유의 (vs PPO 200M)	FlashSAC > PPO(50M)	Holm 유의 (vs PPO 50M)	FlashSAC > FastTD3	Holm 유의 (vs FastTD3)
전체	25	**22**	14	22	16	22	16
고차원	10	**10**	10	10	10	10	7
저차원	15	12	4	12	6	12	9

과제별 최종 값(FlashSAC 평균 ± sd / PPO 200M / PPO 50M / FastTD3 50M):

과제	FlashSAC	PPO 200M	PPO 50M	FastTD3
G1 평지 (Playground)	0.998 ± 0.016	0.585	0.163	0.788
G1 거친 지형 (Playground)	0.853 ± 0.040	0.481	0.006	0.698
T1 평지 (Playground)	0.963 ± 0.012	0.861	0.424	0.874
T1 거친 지형 (Playground)	0.864 ± 0.017	0.466	0.122	0.764
Allegro 재배치 (IsaacLab)	0.866 ± 0.025	0.152	0.049	0.718
Shadow 재배치 (IsaacLab)	0.980 ± 0.016	0.490	0.381	0.962
G1 평지 (IsaacLab)	0.934 ± 0.012	0.663	0.613	0.790
G1 거친 지형 (IsaacLab)	1.029 ± 0.029	0.283	0.098	0.859
H1 평지 (IsaacLab)	0.972 ± 0.007	0.771	0.792	0.786
H1 거친 지형 (IsaacLab)	0.691 ± 0.012	0.591	0.340	0.567
Anymal-C 평지	0.887 ± 0.000	0.723	0.696	0.858
Anymal-D 평지	0.889 ± 0.001	0.712	0.681	0.847
Anymal-C 거친 지형	0.608 ± 0.012	0.439	0.382	0.425
Anymal-D 거친 지형	0.624 ± 0.008	0.454	0.358	0.474
Franka 서랍	0.985 ± 0.008	0.982	0.980	0.998
Franka 들기	0.968 ± 0.004	0.584	0.450	0.756
Franka 당기기	1.000 ± 0.000	1.000	1.000	0.800
Franka 파지 (Genesis)	0.899 ± 0.004	0.851	0.852	0.800
Go2 걷기 (Genesis)	0.919 ± 0.005	0.739	0.776	−0.013
Go2 쉬운 걷기 (Genesis)	0.880 ± 0.003	0.875	0.870	0.314
ManiSkill 컵 집기 (YCB)	0.654 ± 0.101	0.688	0.696	0.516
ManiSkill 구멍 끼우기	0.964 ± 0.034	0.917	0.526	0.000
ManiSkill 컵 밀기	0.198 ± 0.443	0.854	0.895	0.200
ManiSkill 공 굴리기	0.920 ± 0.023	0.833	0.847	0.000
ManiSkill 막대 세우기	0.998 ± 0.004	0.760	0.759	1.000

읽는 법과 예외:

- 고차원 10과제는 전부 PPO 200M을 이기고 Holm 보정 후에도 유의하다. 저차원에서는 15과제 중 12과제에서 평균이 높지만, 유의한 것은 4개뿐이다.
- **예외 1: ManiSkill 컵 밀기(PokeCube).** FlashSAC가 0.198 ± 0.443이다. 5시드 중 한 개만 성공했다. PPO 200M 대비 평균이 낮다(p = 0.037, Holm 보정 후 0.30이라 유의하지 않음). 논문 본문은 이 과제를 따로 다루지 않는다.
- **예외 2: 컵 집기(PickSingleYCB).** FlashSAC가 PPO보다 낮지만 차이는 유의하지 않다(p = 0.70).
- **FastTD3 실패:** 시드 하나 이상이 FlashSAC 평균의 절반 아래인 과제가 7개다. Go2 걷기는 5시드 모두, 구멍 끼우기·공 굴리기는 5시드 모두 0이다. 고차원 10과제에서는 FastTD3 실패가 없다. 본문의 "Go2 Walk와 Franka Pull Cube에서 자주 실패"는 Go2 Walk는 맞고, Franka Pull Cube는 FastTD3 평균 0.8로 실패라고 보기 어렵다(5시드 중 1개 실패).
- **PPO 50M 비교:** PPO를 50M에서 보면 격차가 더 크다. 평지 G1의 PPO 50M 평균은 0.163이다. 같은 50M에서도 FlashSAC가 22과제에서 앞선다. 그래서 "PPO가 계산량 3배를 써서 졌다"는 설명은 샘플 기준으로는 성립하지 않는다.
- **공개 CSV에서 나온 PPO 시드 수:** 과제에 따라 5-6개다. FastTD3는 시드 0-4와 1000-4000을 쓴다.
- **범위 밖 과제 2개:** UnitreeG1PlaceAppleInBowl, UnitreeG1TransportBox는 CSV와 ManiSkill 스크립트에 있지만, 논문 부록 표에 정규화 값이 없어 제외했다.

Figure: `report/figures/fig_gpu_final.png`.

3. 같은 환경 step 비교 요약

위 표의 "PPO 50M" 열과 같다. 같은 50M에서 FlashSAC가 PPO보다 높은 과제는 22개다. 차이가 가장 작은 과제는 Franka 서랍(0.985 대 0.980), 가장 큰 과제는 G1 거친 지형(1.029 대 0.098)이다.

4. CPU 시뮬레이터 (상태 기반), 1M step에서 비교

FlashSAC는 1M step이고 기준선은 각자 더 긴 예산(PPO 4M, TD-MPC2 최대 14M, 나머지 1M)으로 돌렸다. 같은 1M에서 보간해 비교했다.

기준선	시드 수	과제 수	FlashSAC 평균이 높은 과제	Holm 유의	1M 평균 (정규화)
PPO	5	39	37	28	0.259 (FlashSAC 0.872)
SimbaV2	10	39	30	9	0.803
XQC	10	39	30	8	0.791
TD-MPC2	4	39 (검정 34)	31	8	0.647
MR.Q	10	15	13	8	0.765

해석: 평균에서는 FlashSAC가 모든 기준선에 앞선다. 하지만 SimbaV2·XQC·TD-MPC2와의 차이는 39과제 중 8-9개에서만 유의하다. "모든 기준선을 일관되게 앞선다"는 본문 문장은 평균 기준에서만 맞다. PPO와의 차이는 큰 편이다.

과제 예: Ant (1M) FlashSAC 0.749, PPO 0.099, SimbaV2 0.733, XQC 0.751, TD-MPC2 0.469, MR.Q 0.681.

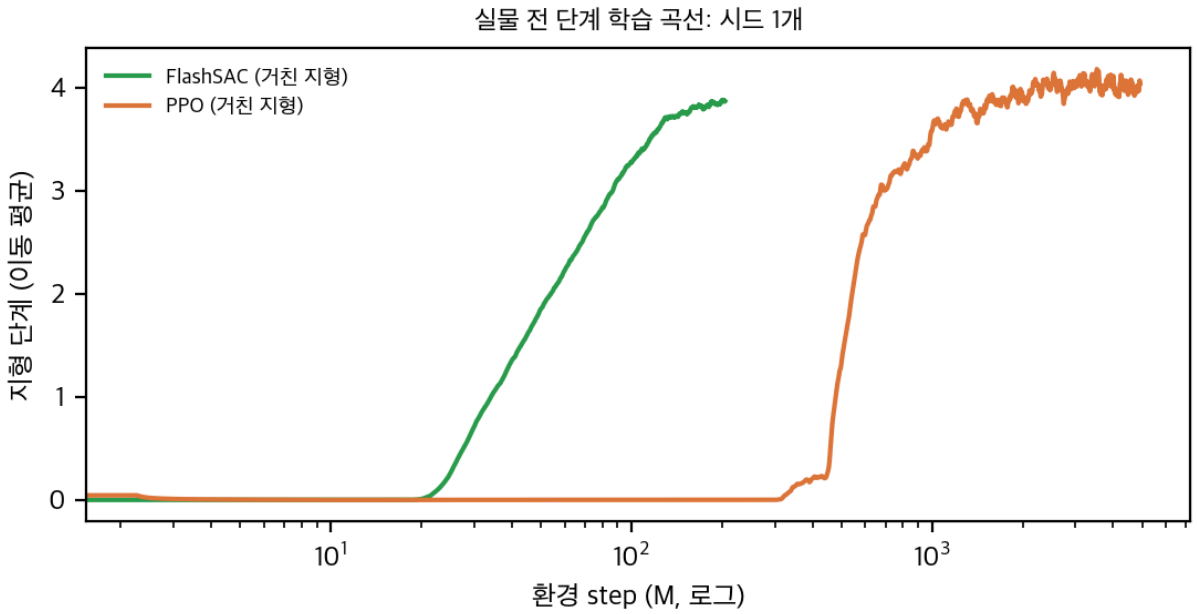
Figure: 표만 있음 (`analysis/recompute_results.json`의 `cpu_state`).

5. 시각 기반 (DMC), 1M step

DMC 10과제 중 나머지 3개(Adroit 계열 이름)는 정규화 값이 없어 뺐다.

기준선	과제 수	FlashSAC 평균이 높은 과제	Holm 유의
DrQ-v2	10	9	6
XQC	8	5	3
DrM	10	5	2
MR.Q	10	**3**	2

해석: DrQ-v2에 대해서는 본문 주장과 맞는다. MR.Q에 대해서는 같은 1M에서 MR.Q가 7과제에서 높다. 본문은 점근 성능에서 "모든 기준선과 같거나 앞선다"고 쓴다. 이 1M 비교에서는 MR.Q에 대해 그 주장이 맞지 않는다. 점근 성능 비교는 기준선의 긴 예산 곡선이 필요해서 여기서는 하지 않았다.

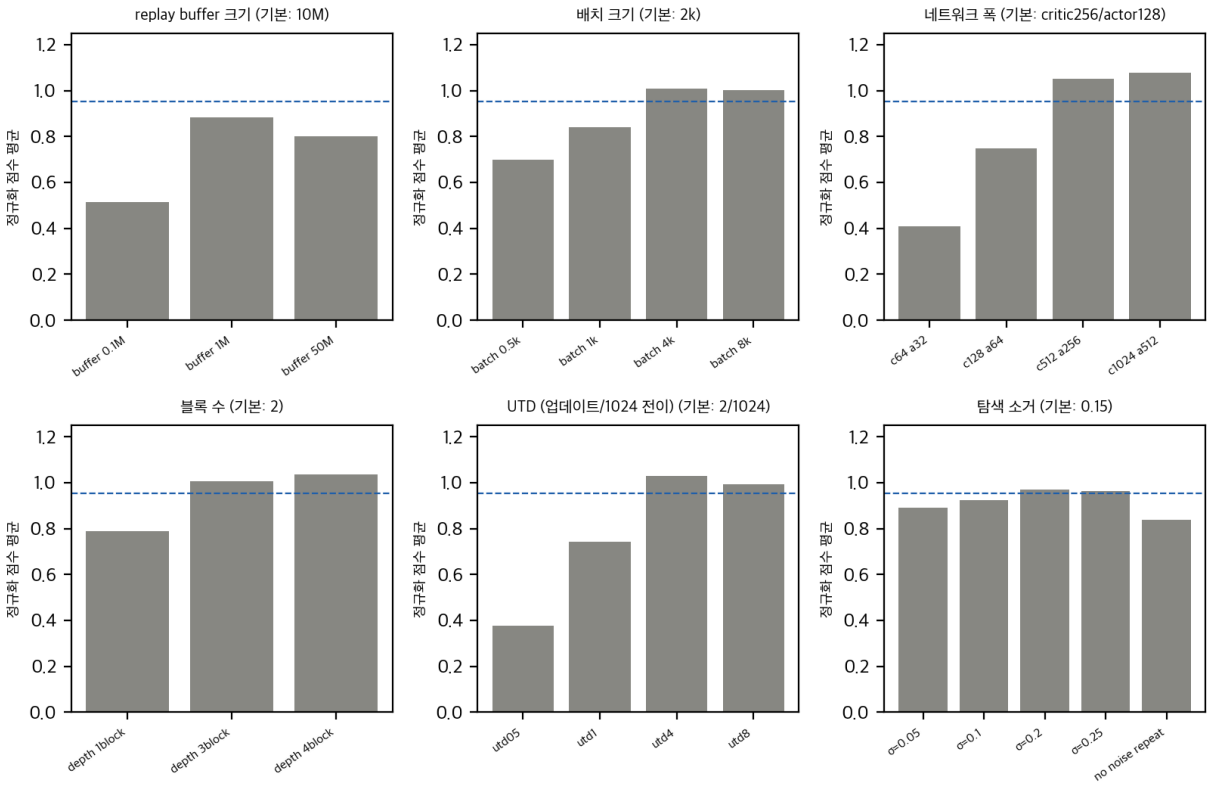


직접 계산: 거친 지형 지형 단계의 이동 평균, 환경 step(로그축). 시드 1개.

6. 실물 전 단계 (G1, 시드 1개)

- 공개 CSV는 시드 42 하나다(학습 곡선 1개). 지형 단계 지표(`avg_terrain_level`) 하나로만 비교한다. 벽시계는 없다.
- **거친 지형:** 지형 단계 3.0에 도달한 환경 step은 FlashSAC 86.7M, PPO 672.8M이다(약 7.8배). 이동 평균 기준 80% 도달 step은 89.4M 대 795.3M이다. 본문의 "4시간 대 20시간"은 계산량이 같은 방향이다(시간 비율 5배, 환경 step 비율 7.8배). 벽시계 자료가 없어 시간 자체는 확인하지 못했다.
- **평지:** 지형 단계가 0이 아닌 3.47(FlashSAC)과 3.41(PPO)에서 시작한다. 그래서 0 기준 도달 시점은 의미가 없다.
- **이상:** 평지와 거친 지형 CSV의 마지막 500점 평균이 같다(FlashSAC 3.869, PPO 4.021). 최댓값도 같다(3.882, 4.184). 두 파일이 같은 실행이었는지, 지표가 상한에 붙었는지 확인이 필요하다. 원본 파일의 md5는 서로 다르다.

Figure: `report/figures/fig_sim2real_rough.png`.



직접 계산: 소거 설정별 정규화 점수 평균(4과제). 점선은 기본값.

7. 소거 실험 (IsaacLab 4과제, 시드 5개)

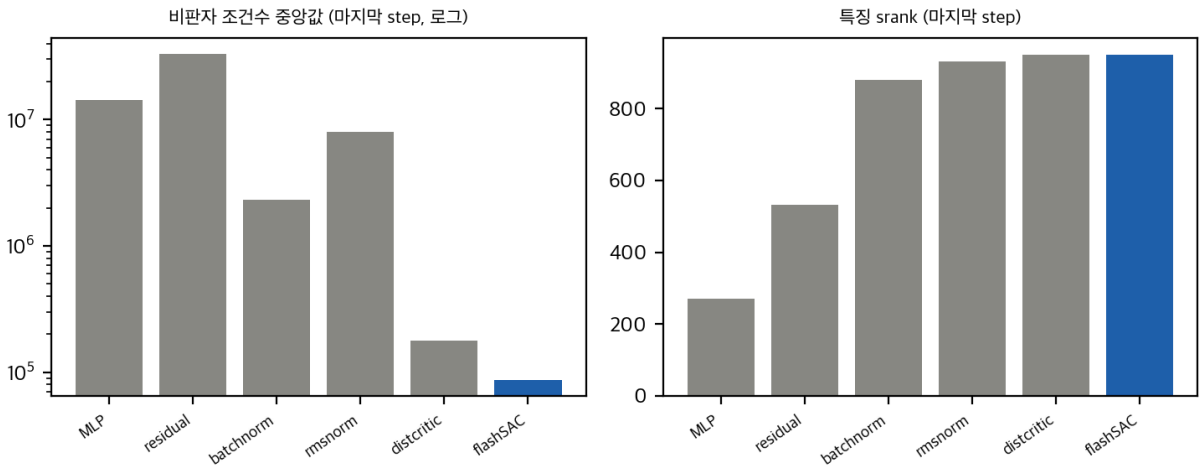
과제는 Shadow·Allegro 재배치, G1 평지·거친 지형이다. 기준은 같은 4과제의 FlashSAC 기본 실행이다(평균 0.952). 값은 정규화 점수 평균(4과제)이다.

축	설정 → 정규화 점수 평균
재생 버퍼	0.1M → 0.515, 1M → 0.883, **10M → 0.952**, 50M → 0.799
배치	0.5k → 0.700, 1k → 0.841, **2k → 0.952**, 4k → 1.009, 8k → 1.003
비판자 폭 / 액터 폭	64/32 → 0.408, 128/64 → 0.750, **256/128 → 0.952**, 512/256 → 1.053, 1024/512 → 1.076
블록 수	1 → 0.790, **2 → 0.952**, 3 → 1.007, 4 → 1.035
UTD (/1024)	0.5 → 0.376, 1 → 0.744, **2 → 0.952**, 4 → 1.027, 8 → 0.992
엔트로피 목표 σ	0.05 → 0.890, 0.1 → 0.923, **0.15 → 0.952**, 0.2 → 0.969, 0.25 → 0.964
잡음 반복	없음 → 0.837 (기준 0.952)

해석 (env step 기준):

- 버퍼는 10M이 최적이라는 본문과 맞다. 50M은 0.799로 떨어진다. 이것은 본문이 "느려진다"고 쓴 내용과 같은 방향이다.
- 배치, 폭, 깊이, UTD는 기본값보다 큰 설정이 env step 기준에서 같거나 높다(4k 이상, 폭 512 이상, 블록 3 이상, UTD 4 이상). 본문은 이런 설정이 **벽시계 기준으로** 빠르다고 말한다. 공개 자료에는 벽시계가 없어서 이 주장은 확인할 수 없다. 기본값이 계산 비용 때문에 고른 값으로 보인다.
- 탐색: σ 0.15-0.2가 평평하게 최적이다(본문과 같음). 잡음 반복을 끄면 0.115 떨어진다. 본문의 주장과 맞는다.

Figure: `report/figures/fig_ablation_scaling.png`.



직접 계산: 비판자 조건수(중앙값, 로그축)와 srank. 마지막 step.

7.1 안정성 지표 (구조 소거, Fig. 9와 비교)

구조 (누적)	조건수 중앙값	조건수 기하평균	srank	죽은 뉴런 비율	특징 크기	기울기 노름
a MLP	1.43e7	1.00e7	271	0.216	6.14	3.78
b +잔차	3.31e7	2.75e7	533	0.162	2.13	10.77
c +BN	2.32e6	4.42e6	880	0	0.52	2.12
d +RMSNorm	7.99e6	1.61e6	932	0	0.55	1.66
e +분포형	1.78e5	1.33e5	950	0	0.55	**0**
f +가중치 정규화 (FlashSAC)	**8.73e4**	1.43e5	950	0	0.51	**0**

본문 주장 대 자료:

- "조건수는 구조를 더할 때마다 단조 감소하고, 최종 구조가 가장 낮다." 자료는 두 번째 부분만 부분적으로 맞는다. 조건수 중앙값과 기하평균 모두 a→b에서 **오르고**(1.43e7 → 3.31e7, 1.00e7 → 2.75e7), 중앙값 기준 c→d에서도 **오른다**(2.32e6 → 7.99e6). 기하평균 기준 e→f는 **소폭 오른다**(1.33e5 → 1.43e5). 최저는 중앙값 기준 f, 기하평균 기준 e다. 단조 감소는 성립하지 않는다.
- 기울기 노름은 e와 f에서 정확히 0으로 기록되었다. 기록 방식의 문제로 보인다(optimizer가 zero_grad 뒤에 값을 기록했을 가능성). 그래서 "기울기 노름이 제한된다"는 주장은 이 자료로 확인할 수 없다. 잔차 블록을 넣으면 기울기 노름이 3.8에서 10.8로 커진다(b). 이것은 본문 Fig. 9d의 MLP·잔차 곡선과 같은 방향이다.
- srank는 e와 f가 정확히 같은 값(949.75)이라 구분력이 없어 보인다.
- 죽은 뉴런은 BN을 넣으면 0이 된다. 이것은 본문의 "포화 방지" 주장과 맞는다.

Figure: `report/figures/fig_stability.png`.

8. 유도와 검산 (`analysis/derivation_checks.py`, 공식 코드를 실행하지 않음)

목표 엔트로피 (식 7). $\bar{H} = \frac{1}{2}|A| \log(2\pi e\, \sigma^2)$, $\sigma = 0.15$. 차원당 −0.478 nats. 행동 차원별 목표는 6 → −2.87, 19 → −9.09, 20 → −9.56, 29 → −13.87, 37 → −17.69. 코드의 엔트로피는 tanh로 눌린 행동의 log π이므로 같은 σ에서 차원당 약 0.022 nats 낮다 (몬테카를로 −0.5006). 이 차이는 작지만, 식 7은 pre-tanh 가우시안 기준이라는 것을 기억해야 한다.

Zeta 잡음 반복. $P(k) \propto k^{-2}$, $k \le 16$. $E[k] = 2.13$. $P(k=1) = 0.63$, $P(k \ge 8) = 0.046$. 잡음을 새로 뽑는 step 비율 = $1/E[k]$ = 0.47. 평균적으로 2.1 step마다 잡음이 바뀐다.

재생 비율 (샘플 기준). 업데이트 수 × 배치 ÷ 새 전이:

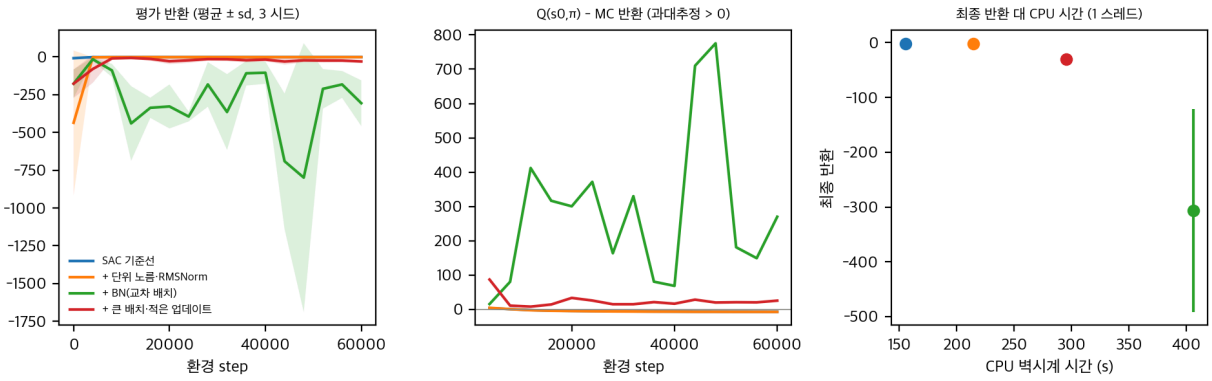
설정	전이당 업데이트	전이당 배치 샘플 수
GPU (1024 env, 업데이트 2, 배치 2048)	0.00195	**4**
CPU (1 env, 업데이트 1, 배치 512)	1	**512**
시각 (1 env, UTD 0.5, 배치 256)	0.5	128

"UTD를 낮췄다"는 말은 GPU에만 맞는다. CPU 설정에서는 전이 하나를 평균 512번 뽑는다. 업데이트 횟수보다 샘플 재사용이 학습을 결정한다는 뜻이다.

학습률 스케줄. 50,000,896 step ÷ 1024 = 48,829 상호작용, 업데이트 97,658회, 액터 업데이트 48,829회. 워밍업 step = int(1e-6 × 97,658) = 0이라 워밍업이 없다.

파라미터 수. 액터 약 0.29M(관측 157, 행동 20 기준), 비판자 앙상블 약 2.25M. 합계 2.54M. 논문 "2.5M"은 이 합계다. 표준 SAC 기준선(0.2-0.5M)과 비교하려면 비판자를 두 개로 세야 한다.

보상 정규화 (식 6) 는 `reward_normalization.py:32–35` 와 맞는다(04 D17).



직접 계산(토이): 평가 반환(평균 ± sd, 3 시드), 과대추정 지표, 최종 반환 대 CPU 시간.

9. SAC 토이: 안정화 요소의 효과 (LQR)

목적. FlashSAC의 안정화 요소가 SAC를 실제로 바꾸는지 CPU에서 확인한다. 공식 코드는 실행하지 않았다. 토이는 이 분석에서 새로 만든 SAC다.

과제. 상태 4차원, 행동 2차원(tanh로 [-1, 1]), 선형 동역학 x' = Ax + Bu + 0.05w(A의 스펙트럼 반지름 0.9, 시드별 고정), 보상 $-(x^{\top}x + 0.1|u|^2)$, 구간 100 step. 병렬 환경 16개, 총 60,000 전이, 시드 3개. 초기 정책의 평균 반환은 −8.73이다.

설정.

설정	내용
base	SAC 기준선. MLP 2×256, ReLU, 배치 256, 환경 step 16개당 업데이트 16회
norm	base + 매 업데이트 뒤 가중치 단위 노름 투영 + RMSNorm 특징
norm_bn	norm + 은닉층 BN(모멘텀 0.01), 현재와 다음 상태를 한 배치로 묶어 계산
fs_sched	norm_bn + 배치 2048, 환경 step 16개당 업데이트 1회 (논문의 "큰 배치, 적은 업데이트" 방향)

Q 과대추정은 $Q(s_0, \pi(s_0))$ – 같은 정책의 600 step 할인 반환(16개 시작 상태)으로 잰다.

결과 (60k 환경 step, 시드 3).

설정	최종 반환 (시드별)	평균 ± sd	업데이트 수	배치	마지막 과대추정 (평균)	CPU 시간 (s)
base	−1.57, −1.90, −1.32	**−1.60 ± 0.29**	58,016	256	−7.2	155
norm	−1.53, −1.95, −1.51	**−1.66 ± 0.24**	58,016	256	−7.5	214
norm_bn	−454.9, −365.2, −99.4	**−306.5 ± 184.9**	58,016	256	**+269.8**	406
fs_sched	−17.8, −38.8, −34.4	**−30.3 ± 11.1**	3,626	2048	+25.3	295

Figure: `report/figures/fig_toy_sac.png`.

해석.

1. **가중치 단위 노름 투영과 RMSNorm(norm)은 이 토이에서 차이가 없다.** 최종 반환은 base와 같고(−1.66 대 −1.60), 과대추정도 같다. 이 과제는 기준선이 이미 안정적이어서 안정화가 필요하지 않았다.
2. **교차 배치 BN(norm_bn)은 세 시드 모두 발산시켰다.** 반환은 −99에서 −455이고, Q 과대추정은 +100에서 +750까지 커졌다. 발산과 과대추정이 같이 나타나므로, 비판자 오차가 쌓이는 방향과 맞는다. 행동 선택을 학습 모드 BN으로 했던 수정 전 버전에서도 시드 0은 −150으로 발산했다(로그에만 남음). 그래서 이 결과는 행동 모드 선택 하나 때문이 아니다.
3. **큰 배치와 적은 업데이트(fs_sched)는 발산하지 않았지만 덜 학습했다.** 업데이트가 3,626회뿐이라 반환은 −30이다. 과대추정이 +25로 남아 있다. 벽시계 시간은 기준선의 약 2배다(배치 2048의 갱신 비용). 즉 이 토이에서는 "적은 업데이트로 같은 성능을 더 빨리"가 나오지 않았다.

한계와 판정. 시드 3개, 과제 1개, 얕은 망, 배치 256 기준이라 논문 설정과 다르다. 그래서 일반화하면 안 된다. 다만 방향은 분명하다. 이 토이 규모에서 BN은 이득이 아니라 해였고, 이것은 02 §3에서 본 DDPG 토이(은닉층 BN이 해로움)와 같은 방향이다. BN 실패의 정확한 원인(배치 크기, 이동 평균과 배치 통계의 차이, 구현)은 아직 가르지 못했다. 배치 2048에서 BN만 쓰는 진단과 평가 모드와 학습 모드 비교가 다음 단계다.

10. 산출 파일

파일	내용
`analysis/recompute_results.py` / `.json`	공개 CSV 재계산 (§2-7)
`analysis/derivation_checks.py` / `_results.json`	유도와 검산 (§8)
`analysis/sac_stabilizer_toy.py` / `sac_stabilizer_toy_results.json`	SAC 토이 (§9)
`analysis/plot_figures.py`	그래프 (`report/figures/fig_*.png`)

FlashSAC 코드 대조 (논문 ↔ 공식 저장소)

0. 대상과 방법

- 저장소: https://github.com/Holiday-Robot/FlashSAC , 커밋 `87edc90` (LICENSE 파일 커밋, 2026-04-09). `repo/` 에 `--depth 1` 로 얕게 복제했다. 라이선스 MIT. 커밋 이력은 1개뿐이다.
- 방법: **정적 읽기만 했다.** 코드를 실행하지 않았다(학습, 테스트, `uv sync`, 설정 로드 모두 안 함). 논문 식과 부록 표를 코드 줄과 대조했다.
- 줄 번호는 위 커밋 기준이다. 파일 경로는 `repo/` 기준이다.
- 파라미터 수와 기타 값은 코드의 층 정의를 손으로 세어 `analysis/derivation_checks.py` 에서 계산했다(03 §8).
- 공개 학습 곡선 CSV(`repo/results/`)는 재계산에 썼다(03).

1. 대조 표 (D1-D30)

판정: **일치**는 논문 설명과 코드가 같다. **차이**는 코드와 논문 본문·표가 다르다. **문서 불일치**는 논문 안에서 서로 다르다. **미기재**는 논문에 없고 코드에 있다. **부재**는 논문에 있으나 공개 코드에 없다.

1.1 목적 함수와 학습 루프

#	항목	논문	코드 (줄)	판정
D1	비판자 손실	식 (4)는 MSE. §4.2는 분포형 비판자와 교차 엔트로피	`flash_rl/agents/flashSAC/update.py:249` `ce_loss = -(target_probs · pred_log_probs).sum` . 출력은 `layer.py` `EnsembleCategoricalValue` 의 101 bin softmax	문서 불일치 (식 표기만 다름). 구현은 §4.2 설명과 일치
D2	최소 이중 Q	식 (5)의 min Q	`update.py:15–31` `_select_min_q_log_probs` : 샘플마다 기대 Q가 작은 쪽 비판자의 분포를 고른다	일치 (기대값 기준 min)
D3	목표값	식 (5): r + γ(min Q − α log π)	`update.py:56` `target_bin_values = reward + gamma*(bin_values - actor_entropy)*(1-done)` , `actor_entropy = α·log π` . `update.py:57` 지지 [min_v, max_v]로 자른다	일치. 지지로 자르는 단계는 논문에 없다(미기재)
D4	n-step	표 9: n = 1 (GPU). 표 11: 시각 n = 3	`scripts/run_isaaclab.sh:48` `n_step=3` . 다른 GPU 스크립트(`run_maniskill.sh` , `run_genesis.sh` , `run_mujoco_playground.sh`)와 `configs/agent/flashSAC.yaml` 의 기본값은 1. 버퍼의 n-step 반환은 `torch_buffer.py:103–132`	문서 불일치: IsaacLab만 n=3
D5	종료와 절단	명시 없음	목표는 `terminated` 만 쓴다(`update.py` 배치 키). 보상 정규화의 누적 반환 리셋은 `terminated or truncated` (`reward_normalization.py` `_update_reward_stats`)	미기재 (표준적 처리)
D6	블록 구조	그림 2: Linear → BN → ReLU → Linear → BN → ReLU → (+잔차). 본문 "pre-activation BN"	`layer.py:91–107` `FlashSACBlock` : `w1 → norm1 → relu → w2 → norm2 → relu → +residual` . 그림과 같다	일치. 본문의 "pre-activation"은 명칭 혼동: BN은 선형 뒤, ReLU 앞이다

#	항목	논문	코드 (줄)	판정
D7	최종 정규화	RMSNorm으로 특징 노름을 제한	`network.py` `post_norm = UnitRMSNorm` 후 `predictor`. `layer.py:62–75` : `F.rms_norm`, 가중치 노름 √d	일치
D8	가중치 투영	각 가중치 벡터를 단위 노름, (γ, β)는 노름 √d(본문은 "d"로 보이나 PDF 기호 손실)	단위 행 투영: `layer.py:19–24` (`F.normalize`, 입력 차원 기준). BN (γ,β): `layer.py:52–57` (노름 √d). 호출은 `update.py:155` (액터), `update.py:269` (비판자), `agents/utils/network.py` `normalize_parameters`. 온도 파라미터는 투영하지 않음	일치 (√d 여부는 PDF 추출로 확인 불가; 코드는 √d)
D9	교차 배치 (비판자)	현재와 다음 상태를 한 배치로 묶어 BN 통계 공유	`update.py:214` `obs_all = cat(obs, next_obs)`, 표적·예측 비판자 모두 이 배치로 계산	일치
D10	교차 배치 (액터 갱신)	논문 본문은 비판자 쪽만 말함	`update.py:105` `actor_obs_all = cat(obs, next_obs)` 로 액터를 통과시킨 뒤 앞 절반만 씀. 그래서 액터 BN 통계는 다음 상태 포함 2B 샘플로 계산됨	미기재 (코드 특이점)
D11	액터 갱신 때 비판자 모드	명시 없음	`update.py:116–123` : `critic(..., training=False)`. 액터 손실의 Q는 BN 이동 평균을 쓰고, 표적·예측 비판자 호출 (`update.py:219`, `:242`)은 `training=True` 로 배치 통계를 씀	미기재 (모드가 다름)
D12	표적 비판자 EMA	식 (3), τ = 0.01	`agents/utils/network.py` `_ema_update_fn` : `torch._foreach_lerp_(params, src, τ)`. 매개변수만 갱신하고 BN 버퍼(이동 평균)는 갱신하지 않음	일치 (버퍼 제외는 미기재)
D13	액터 지연 갱신	표 9: update delay 2	`agent.py:501` `do_actor_update = (step % actor_update_period == 0)`	일치
D14	온도와 목표 엔트로피	식 (7): $\bar{H} = ½$	A	$\log(2\pi e\sigma^2)$, σ = 0.15
D15	잡음 반복	§4.3: 길이 k 동안 ε 고정. "짧은 로컬 상태"	`agent.py:245` `reinit = (cur_count == 0)	(cur_count >= cur_n) . 카운터와 길이 cur_n 은 스칼라 텐서라 **모든 환경이 같은 시점에 갱신**. 잡음 벡터는 randn_like(mean) 로 환경마다 뽑힘(agent.py:247`)
D16	Zeta 분포	P(k) ∝ k^(-s), s = 2, 최대 16	`agent.py:394` `_build_truncated_zeta_cdf(mu=2.0, max_n=16)` ; `agent.py:206` `pmf = ns ** (−mu)`	일치. E[k] = 2.13, 매 갱신 비율 0.47 (03 §8)
D17	보상 정규화	식 (6): $\bar{r} = r / \max(\sqrt{(\sigma^2_G+\varepsilon)}, G_max,t / G_max)$	`reward_normalization.py:32–35` `denominator = max(sqrt(G_var+eps), G_r_max/G_max)` ; `G_r_max` 는 지금까지	G_r
D18	학습률	표 9: 코사인 감쇠 3e-4 → 1.5e-4	`configs/agent/flashSAC.yaml` : 워밍업 비율 1e-6 → `int(1e-6 × 97,658) = 0` 이라 워밍업 없음. 감쇠 길이는 전체 업데이트 수	일치 (워밍업 없음은 미기재)

#	항목	논문	코드 (줄)	판정
D19	기울기 클리핑	§4.2: "기울기 노름을 명시적으로 제한"	`grep clip_grad` 결과 0건. 기울기 노름은 가중치 투영을 통해서만 간접 제한	차이 (표현이 과함)
D20	혼합 정밀도	본문: 벽시계 5-10% 감소	GPU 설정 `use_amp: true` (`run_isaaclab.sh`): `autocast(fp16)` + `GradScaler` (`update.py`). 통계는 float32(`layer.py` 주석). CPU 설정은 `use_amp=false`	일치. 5-10% 수치는 코드에 측정 코드 없음 (D29)
D21	컴파일	§4.1: JIT	`agent.py` `_resolve_compile_mode` : torch ≥ 2.9는 `max-autotune`, 그 아래는 `reduce-overhead`. README는 Python 3.11에서 2.9.1을 쓴다고 함	일치 (설치 환경 종속)
D22	병렬 환경·버퍼	1024 env, 버퍼 10M	`run_isaaclab.sh` : `num_train_envs=1024`, `buffer_max_length=10_000_000`, `buffer_device_type='cuda'`, `buffer_min_length=100_000`. 기본 설정은 10k. `train.py:123–125` : 버퍼가 찰 때까지 무작위 행동	일치. 최소 버퍼 크기(100k)와 무작위 시작은 논문에 없음 (미기재)
D23	UTD	본문: 2/1024. 표 9: "2/2048"	`run_isaaclab.sh` : `updates_per_interaction_step=2`, 상호작용 1회에 전이 1024개(`train.py:121` `env_step = interaction_step * num_train_envs`). 즉 2/1024	문서 불일치 (표 9 오기로 보임). 코드는 본문과 같다
D24	배치 크기	GPU 2048, CPU 512, 시각 256	GPU 스크립트 2048, CPU 스크립트 512(`run_mujoco.sh` 등). 시각 설정 없음 (D26)	일치 (시각 제외)
D25	시드	명시 없음	스크립트 시드 `(0 1000 2000 3000 4000)`, 공개 CSV도 같은 5개 (FlashSAC 전부)	미기재 (논문은 개수를 적지 않음)
D26	시각 기반 학습	§5.3: 3층 합성곱 인코더, 프레임 스택 3, 3단계 반환, 액션 반복 2	픽셀 관련 코드 0건(`from_pixels`, `conv`, `frame_stack` 검색). `dmc.py` 는 상태 관측만 쓴다. `run_dmc.sh` 는 배치 512, n = 1로 상태 기반 설정	**부재**: 공개 코드에 시각 설정이 없다. 시각 CSV(`visualRL/`)는 있다
D27	평가 빈도	학습 곡선	`configs/flashSAC_base.yaml` : 평가와 로그 주기 = 총 상호작용 수 / 10. 곡선은 실행당 11점. GPU는 약 5M step 간격, CPU는 약 0.1M 간격	미기재 (곡선 해상도)
D28	평가 방식	명시 없음	평가는 결정적 행동 `tanh(mean)` (`agent.py:241`), 평가 에피소드 1024개(GPU), 50개(CPU)	미기재
D29	벽시계 측정	부록 A: 환경 시간 + 업데이트 시간 합산, 업데이트 시간은 한 과제에서 측정해 재사용	`train.py`, `flash_rl/common/logger.py`, `flash_rl/evaluation.py` 에 시간 측정 호출 없음. 공개 CSV에도 시간 열이 없다	**부재**: 벽시계 측정 코드가 공개되지 않음
D30	실물 배포	§5.4, 부록 D: PD 제어 200 Hz, Unitree SDK, ROS 2 노드, Pybind	배포 코드 없음. `uv.lock` 과 CSV, ManiSkill 스크립트의 과제 이름에만 Unitree 문자열 있음	**부재**: 실물 배포 코드 없음. 학습 환경(IsaacLab 설정)만 있음

1.2 설정 값 요약

항목	논문 (표 9-10, 본문)	공개 스크립트 (GPU / CPU)
환경 수	1024 / 1	1024 / 1
배치	2048 / 512	2048 / 512
UTD (업데이트/상호작용 단위)	2/1024 (표 9는 2/2048)	2 / 1

항목	논문 (표 9-10, 본문)	공개 스크립트 (GPU / CPU)
버퍼	10M / 1M	10M / 1M
블록 수 (액터, 비판자)	2, 2	2, 2 (`configs/agent/flashSAC.yaml`)
은닉 폭 (액터, 비판자)	128, 256	128, 256
이중 비판자, 원자 수	2, 101	2, 101
표적 τ, 액터 지연	0.01, 2	0.01, 2
n-step	1 (표 9), 3 (시각)	**3 (IsaacLab 스크립트)**, 1 (나머지)
감마	IsaacLab 0.99, Playground 0.97	스크립트별 0.99
학습률	3e-4 → 1.5e-4 (코사인)	같음
σ_tgt, 온도 초기	0.15, 0.01	같음
ζ 지수, 최대 반복	2, 16	같음
시드	(미기재)	0, 1000, 2000, 3000, 4000

1.3 파라미터 수 (손으로 셈, 03 §8)

항목	액터	비판자 앙상블(2개)	합계
Shadow Repose (관측 157, 행동 20)	290,402	2,251,150	2,541,552
G1 평지 (관측 123, 행동 37)	290,368	2,242,378	2,532,746
HalfCheetah (관측 17, 행동 6)	268,590	2,171,686	2,440,276

논문 §4.1의 "2.5M 파라미터"는 **액터와 비판자 앙상블을 합친 값**과 맞는다. 액터만은 0.3M 남짓이다. 표준 SAC 기준선(0.2-0.5M)과 비교할 때는 비판자 두 개를 함께 세어야 같은 기준이다.

2. 재현 검증이 하지 않은 것 (정적 읽기의 한계)

- 학습 곡선과 최종 점수가 코드에서 나온다는 것은 확인하지 못했다. 공개 CSV는 스크립트와 같은 시드, 같은 과제 이름을 쓰지만, 실행 기록을 대조하지는 않았다.
- 수치 안정성(부동소수점, torch.compile 결과, AMP 손실 스케일링)은 실행하지 않아서 확인하지 못했다.
- 시각 설정(D26)과 실물 배포(D30)는 코드가 없어서 검증할 수 없었다.

3. 구조 요약 (코드 기준 흐름)

1. `train.py` 루프: 상호작용 1회에 `process_transition` → 버퍼가 찼으면 `update_counter += updates_per_interaction_step` 만큼 `agent.update()`.
2. `agent.py` `update()`: 버퍼에서 배치 샘플 → 보상 정규화(식 6) → `_update_networks`: 액터(2회마다 1회) → 온도 → 비판자 → 표적 EMA.
3. `update.py` `update_critic`: 교차 배치로 표적 분포 계산(`_compute_categorical_td_target`) → 교차 엔트로피 → Adam 갱신 → 가중치 투영.
4. 샘플링: `sample_actions`는 BN 이동 평균으로 평균·표준편차를 얻고, `temperature=1`로 잡음을 곱해 행동을 만든다. 잡음은 Zeta 길이만큼 유지된다.

4. 구현 판정 (재현 가능성)

판정: 공식 재현은 이 맥에서 불가능하다. 알고리즘을 자기 설정에 옮기는 것과 CPU 소규모 검증은 가능하다.

항목	필요 자원	이 맥(Apple M4 Pro, 약 24 GiB 통합 메모리, MPS, CUDA 없음)에서	노력	판정
공식 GPU 재현: IsaacLab, MuJoCo Playground, ManiSkill, Genesis 25과제 × 시드 5	RTX 30/40/50 계열 또는 Blackwell GPU (README), Ubuntu 22.04/24.04, Python 3.10/3.11, IsaacLab 설치	CUDA가 없다. 저장소가 CUDA 경로를 전제하고, `buffer_device_type='cuda'`. MPS로는 같은 설정을 돌릴 수 없다	큼. 대략 과제당 30-90 min × 시드 5 × 25과제 (논문 Fig. 3 벽시계 축 기준) ≈ 100 GPU시간 이상	**불가**
공식 CPU 재현: MuJoCo v4 / DMC / MyoSuite / HumanoidBench, 1 env, 배치 512	Python 3.10/3.11, uv, gymnasium, mujoco (README는 MuJoCo 2.1 "mujoco210" 필요), dm_control, myosuite, humanoid-bench	`.venv`에 torch 2.15와 mujoco 4.0.0이 있으나 gymnasium은 없다. 설치는 승인 필요	중간. 토이 측정(2 ms/업데이트, 16 업데이트/iter)에서 외삽하면 실행당 약 1-2시간(1M 업데이트, 배치 512)	**조건부**: 패키지 설치 승인 후 과제 1-2개, 시드 1-2개 정도
시각 DMC 재현	GPU, 렌더링(EGL), 시각 인코더 코드	공개 저장소에 시각 설정 코드 없음 (D26)	큼	**불가** (코드 없음)
실물 G1 배포	Unitree G1, GPU 학습, ROS 2, Unitree SDK	로봇 없음. 배포 코드 없음 (D30)	매우 큼	**불가**
알고리즘 재구현 (PyTorch, 이 저장소 구조 참고)	없음 (CPU와 MPS 가능)	토이로 확인 (03 §9). 교차 배치, 가중치 투영, 분포형 비판자는 약 300줄	중간	**가능**
공개 CSV 재계산 (정규화, 유의성, 소거 지표)	없음	완료 (03)	완료	**완료**

그래서 실제로 할 수 있는 것은 두 가지다. (a) 공개 CSV와 논문 표로 주장 점검(이 분석에서 완료), (b) CPU에서 알고리즘 요소를 작은 과제에 옮겨 보기(토이 완료). 공식 GPU 재현은 CUDA 장비가 있어야 하고, 실물 배포는 로봇이 있어야 한다.

5. 검증 공백과 이 코드를 믿기 전에 확인할 것

1. 벽시계 시간 측정 코드가 없으므로 "수 시간에서 수십 분"과 "3배 계산량" 주장은 코드로 확인할 수 없다(D29).
2. 시드 개수와 시드 선택이 논문에 없다. 공개 CSV가 5개 시드로 맞으면 공식 결과와 대조할 수 있다(D25).
3. 개수와 설정 표가 맞지 않는다: UTD(D23), n-step(D4), 과제 개수(03 §1).
4. 액터 갱신에서 BN 통계에 다음 상태를 넣고 비판자를 평가 모드로 쓰는 조합(D10, D11)은 논문이 다루지 않는다. 재구현할 때 같은 선택을 할지, 본문대로 할지 정해야 한다.
5. 기울기 클리핑이 없으므로 기울기 폭주가 생기면 가중치 투영 말고는 막을 장치가 없다(D19). 이 점은 소거 실험에서 기울기 노름이 0으로 기록된 것과 같이 확인이 필요하다(03 §7).